

Project Design Phase

Solution Architecture

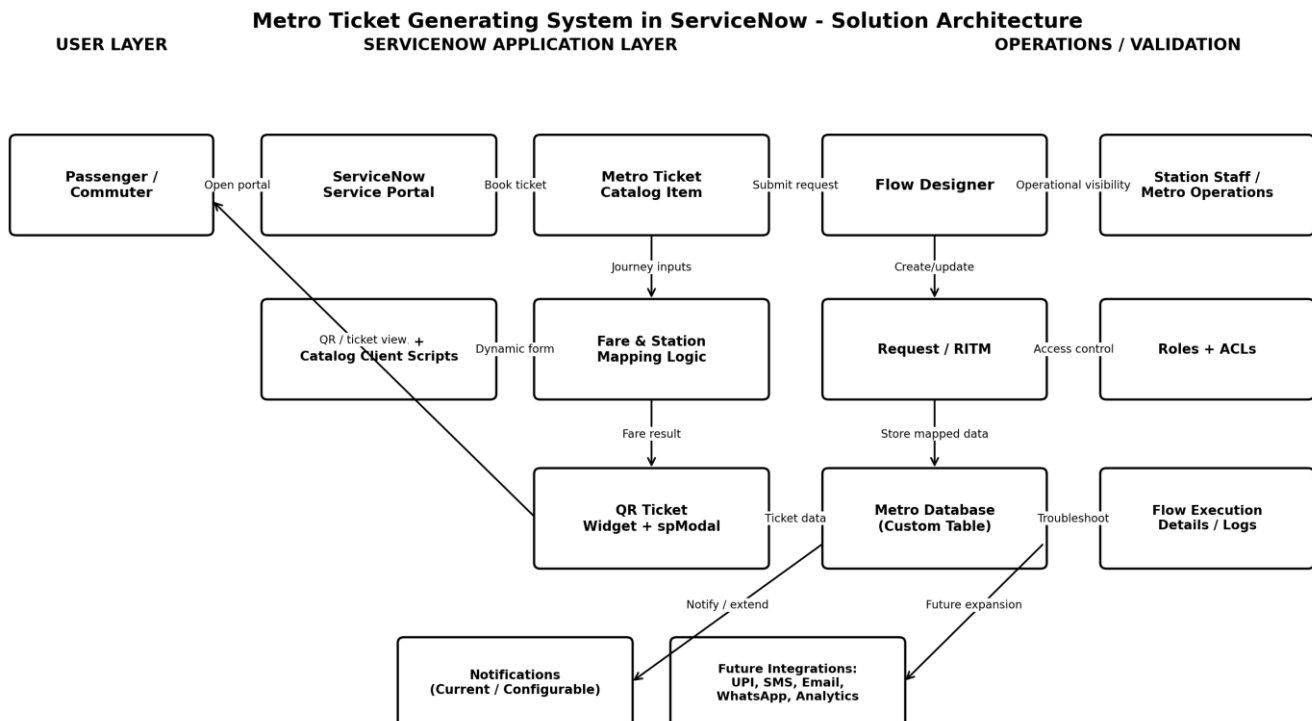
Date	29 September 2026
Project Name	Metro Ticket Generating System in ServiceNow
Maximum Marks	4 Marks

Solution Architecture:

The solution architecture bridges the identified metro-ticketing problems with the ServiceNow technologies used to implement the solution. It defines how passenger inputs move through the portal, catalog item, fare logic, Flow Designer, request records, custom Metro Database table, and QR ticket presentation, while also showing how station staff and administrators interact with the operational side of the system.

- Solve the business problem of slow, manual, and paper-dependent metro ticket booking through a centralized digital workflow.
- Describe the structure and behavior of the ServiceNow solution from passenger booking to ticket verification.
- Define the major functional blocks used in the current implementation and show how data moves between them.
- Support maintainability through Flow Designer-first automation, minimal client-side scripting, and clear separation of user, application, data, and operational responsibilities.

Solution Architecture Diagram



Architecture Components and Responsibilities

S.No.	Component	Architecture Role	Responsibility
1	Passenger / Commuter	Primary end user who initiates metro ticket booking and receives the digital ticket.	Enters journey details, checks fare, submits request, tracks status, presents QR ticket.
2	ServiceNow Service Portal	Passenger-facing access layer for the digital metro service.	Hosts the booking experience and displays the request / ticket information.
3	Metro Ticket Catalog Item	Structured booking form implemented using Service Catalog.	Captures Source, Destination, Passenger Type, Number of Tickets and other booking inputs.
4	UI Policies & Catalog Client Scripts	Client-side behaviour layer used only where required.	Controls visibility, validation and dynamic population of form fields.
5	Fare & Station Mapping Logic	Business logic used to derive journey fare from station selections and ticket quantity.	Calculates and validates the fare before / during booking processing.
6	Flow Designer	Main automation engine of the solution.	Processes submitted requests, maps variables, creates/updates records, triggers tasks and downstream actions.
7	Request / RITM	Native ServiceNow request-management records.	Maintains the service-request lifecycle and booking status context.
8	Metro Database (Custom Table)	Structured persistent storage for metro ticket / booking data.	Stores mapped booking fields for operational use and ticket verification.
9	QR Ticket Widget + spModal	Portal-compatible ticket presentation layer.	Displays the generated digital ticket / QR information reliably within Service Portal.
10	Station Staff / Metro Operations	Operational users responsible for validation and monitoring.	View ticket information and support ticket validation or operational actions.
11	Roles & ACLs	Security and access-control layer.	Restricts record and operational access according to authorized roles.
12	Flow Execution Details / Logs	Troubleshooting and observability mechanism.	Used to trace failures, validate condition paths, inspect inputs/outputs and resolve workflow issues.

End-to-End Data and Control Flow

1. Passenger opens the ServiceNow Service Portal and accesses the Metro Ticket Catalog Item.
2. The user selects Source, Destination, Passenger Type, and Number of Tickets.
3. UI Policies and required Catalog Client Scripts control dynamic form behaviour and input validation.
4. Fare and station mapping logic determines the applicable ticket amount.
5. The passenger submits the booking request.
6. Flow Designer processes the request, reads the catalog variables, and creates or updates the required records.
7. Booking information is mapped into the custom Metro Database table while the request/RITM maintains workflow status.
8. QR ticket information is prepared and displayed through the portal-compatible widget / spModal mechanism.
9. The passenger presents the digital QR ticket to station staff.
10. Station staff or Metro Operations verify the ticket information according to their authorized access.

11. Flow Execution Details and logs are used by administrators to investigate fare, variable-mapping, or automation failures when required.

Architecture Design Principles

Principle	Application in this Project
Low-code first	Flow Designer is the primary automation mechanism, with custom scripting kept minimal.
Separation of concerns	Portal/UI, business logic, workflow processing, storage, security, and validation are treated as distinct architectural responsibilities.
Centralized data	Ticket data is stored in structured ServiceNow records instead of remaining only inside the visible catalog form.
Role-based security	ACLs and roles protect operational data and limit access according to user responsibility.
Maintainability	The architecture is designed so future ServiceNow administrators can understand and modify the workflow.
Extensibility	Payment, SMS/email ticketing, WhatsApp services, and analytics can be added later without replacing the core booking workflow.

Current Boundary and Future Expansion

The current architecture covers the core booking, fare-processing, workflow, storage, QR-ticket, request-tracking, security, and troubleshooting functions implemented in ServiceNow. Payment gateway / UPI integration, SMS or email ticket delivery, WhatsApp-based booking, and travel analytics are architectural extension points and are not presented as completed functions in the current project.